Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 03 Notes
UML Class Design + Package Structure

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **3**
5.1 UML class diagram notation (quick cheat sheet) . . . 3
5.2 Step-by-step: find classes from use cases . . . 3
5.3 Responsibilities (avoid God class) . . . 3
5.4 Relationships (has-a vs is-a) . . . 4
5.5 Interfaces/abstractions to reduce coupling . . . 4
5.6 Worked mapping: "Create Booking" to classes/methods . . . 4
5.7 Package structure (recommended) . . . 4

6 Mini Example (Campus resource booking class list) **5**

7 Deliverable-2 checklist **5**

8 In-Class Activity (evidence) **5**

9 In-Class Exercises (with brief solutions) **5**

10 Self-Study Practice (no solutions) **6**

11 Checkpoint Questions (with sample answers) **6**

1 Lecture Overview

Now that you have use cases, you must design the class structure of the system. This lecture focuses on:

- identifying classes and responsibilities,
- connecting classes with correct relationships,
- reducing coupling using interfaces/abstractions,
- organizing code into packages.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: UML class diagrams; package/module structure; design for maintainability

3 Learning Outcomes

After this lecture, you should be able to:

- Convert use cases into a draft UML class diagram (entities + services + UI + DAO).
- Identify relationships (has-a, is-a) and responsibilities (avoid God classes).
- Propose a clean package structure for your project.
- Complete Deliverable-2: class diagram (draft) + package structure.

4 Key Terms (Quick)

- **Entity/model**: represents real-world data (Resource, Booking, User).
- **Service**: business rules (booking validation, overlap checks).
- **DAO**: database access layer (CRUD + SQL).
- **UI layer**: Swing screens/panels.
- **Association**: two classes related (Booking uses Resource).
- **Composition**: strong has-a (Booking has Resource reference).
- **Inheritance**: is-a relationship (use carefully).
- **Coupling**: how strongly modules depend on each other.

5 Detailed Notes

5.1 UML class diagram notation (quick cheat sheet)

A UML class diagram is a **communication tool**. In this course, we use it to:

- show important classes and their responsibilities,
- show relationships between classes (especially in your domain model),
- justify separation of UI/service/DAO layers.

Common notations you should recognize:

- A class is drawn as a box with compartments: **Name**, **Attributes**, **Methods**.
- Visibility markers: `+` public, `-` private, `#` protected.
- Association: a line between classes ("uses/has-a").
- Multiplicity: `1`, `0..1`, `*`, `1..*` (how many objects).
- Inheritance (is-a): arrow with a hollow triangle pointing to parent.
- Composition (strong has-a): filled diamond at the "owner" side (use only when lifecycle is tightly linked).

Tip: do not try to include every getter/setter in the diagram. Focus on the design.

5.2 Step-by-step: find classes from use cases

A practical method:

1. List core use cases (from Deliverable-1).
2. Extract nouns → candidate entities (Resource, Booking, User).
3. Extract verbs → operations (create booking, cancel booking).
4. Group operations into services (BookingService).
5. Identify persistence needs → DAOs (BookingDao, ResourceDao).
6. Identify UI screens/panels (AddResourcePanel, BookingPanel, ListBookingsPanel).

5.3 Responsibilities (avoid God class)

Each class should have one main responsibility. Bad sign:

- One class does UI + SQL + validation + printing.

Better separation:

- UI reads input and calls service.

- Service enforces rules and calls DAO.
- DAO only performs DB operations.

5.4 Relationships (has-a vs is-a)

- Prefer composition (has-a) for most domain relationships.
- Use inheritance only for true is-a (rare in small projects).
- Interfaces are useful to reduce coupling (e.g., DAO interface).

5.5 Interfaces/abstractions to reduce coupling

Examples:

- `ResourceRepository` interface with methods `save/findAll`.
- Implementation `JdbcResourceRepository` uses JDBC.
- UI/service depends on interface, not implementation (easy to replace/mock).

5.6 Worked mapping: "Create Booking" to classes/methods

To make the class diagram practical, map one use case to the methods you will implement:

- **UI layer:** `CreateBookingPanel.onSaveClicked()`
- **Service layer:** `BookingService.createBooking(...)`

  validations: mandatory fields; start < end; overlap rule.
- **DAO layer:** `BookingDao.insert(...)` and `BookingDao.findOverlaps(...)`
- **Model:** `Booking` object carries data between layers

Outcome: your class diagram should clearly show where each responsibility lives.

5.7 Package structure (recommended)

Keep a simple layered package structure:

- `com.upes.crm.model` — entities
- `com.upes.crm.dao` — JDBC DAOs
- `com.upes.crm.service` — business rules
- `com.upes.crm.ui` — Swing UI
- `com.upes.crm.util` — helpers (DB connection, validation)

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project:

Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

6 Mini Example (Campus resource booking class list)

Entities:

- `Resource(id, name, type, capacity)`
- `Booking(id, resourceId, userId, startTime, endTime, status)`
- `User(id, name, role)`

Services:

- `BookingService`: overlap checks; validate time range; call DAO.

DAOs:

- `ResourceDao`, `BookingDao`, `UserDao`

UI:

- `MainFrame`, `AddResourcePanel`, `CreateBookingPanel`, `ListBookingsPanel`

7 Deliverable-2 checklist

- UML class diagram draft (10–20 classes is typical for this project size)
- Show key relationships (especially Booking → Resource)
- Package structure documented (folder tree or package list)
- Updated issues: one issue per major class/module group (optional)

8 In-Class Activity (evidence)

- Upload class diagram and package structure to repo.
- Create PR if you add initial project skeleton folders/packages.
- Assign issues to teammates based on modules (UI/DB/service).

9 In-Class Exercises (with brief solutions)

1. From your use cases, list 5 nouns (candidate classes).

   Sample: Resource, Booking, User, Room, InventoryItem (depends on domain).

2. For one use case, list 3 methods that must exist.

   Sample: `createBooking`, `findOverlaps`, `insert`.

3. Which layer should contain overlap validation: UI, service, or DAO?

   Answer: service layer (business rule), with DAO providing data to check overlaps.

10 Self-Study Practice (no solutions)

1. Draw a draft UML class diagram for your project (10–20 classes is typical).
2. Write a package structure for your code (model/dao/service/ui/util).
3. Choose 2 use cases and map them to UI/service/DAO methods (like the worked mapping).

11 Checkpoint Questions (with sample answers)

Q1. Why separate UI, service, and DAO layers?

Sample answer: Separation reduces coupling and makes code easier to maintain and test. UI changes should not require SQL changes, and DB changes should not break UI logic directly.

Q2. What is the benefit of interfaces in this design?

Sample answer: Interfaces let you program to a contract, so you can swap implementations (e.g., JDBC vs in-memory) and reduce dependency on concrete classes.